Canadian Wildfires in 2023

NFDB points - Data Cleaning

This notebook cleans the National Fire Database (NFDB) point data. As a result one gets a filtered, classified dataset of uncontrolled wildfires in Canada for the year 2023.

Short summary of the following steps > 0. Import & Constants > 1. Preparation and Cleaning > 2. Filtering > 3. Classification of fire size > 4. Weighting > 5. Export of processed data

0. Import & Constants

```
from pathlib import Path
import pandas as pd

#With pathlib the storage paths stay portable across any device
#Using ../ we go up one level (out of notbook folder) to reach the
#project root and then enter the data folder

DATA_DIR = Path("../data")
RAW_PATH       = DATA_DIR / "raw"       / "NFDB_point.csv"
PROCESSED_PATH = DATA_DIR / "processed" / "fire_2023_clean.csv"
```

a) Analysis parameters

```
TARGET_YEAR = 2023

#Canada lies entirely in the western hemisphere,
#so any positive longitude is definitively a data entry error
MAX_VALID_LONGITUDE = 0
MIN_VALID_LATITUDE = 0

#Size class thresholds in hectars
# Minimum value observed: 200ha | Maximum value observed: 885388ha
SMALL_HA = 500
MEDIUM_HA = 1000
BIG_HA = 10000
VERY_BIG_HA = 100000

#Exponential weighting creates strong visual contrast between classes
CLASS_WEIGHTS = {
    "small": 1,
    "medium": 2,
    "big": 4,
    "very big": 16,
    "mega fire": 256
}

#preserving logical and not alphabetical order
CATEGORY_ORDER = ["small", "medium", "big", "very big", "mega fire"]
```

b) Load Raw Data

```
#You may have to take into account that when loading the data yourself
#from the readme then the seperator may have been changed to a comma

fire_df = pd.read_csv(RAW_PATH, sep=";")

assert len(fire_df) > 0, "Loaded file is empty."
print(f"Loaded {len(fire_df)} records from raw data.")
```

```
Loaded 15206 records from raw data.
```

1. Preparation and Cleaning

a) Renaming the columns to snake_case

```
#Only the columns relevant for later are renamed
NEW_NAMES = {
    "SRC_AGENCY": "province",
    "LONGITUDE": "longitude",
    "LATITUDE": "latitude",
    "SIZE_HA": "size_ha",
    "FIRE_ID": "fire_id",
    "FID": "fid",
    "CAUSE": "cause",
    "YEAR": "year"
}

fire_df = fire_df.rename(columns=NEW_NAMES)
```

b) Droping not used columns

```
COLUMNS_DROP = [
    "the_geom", "NFDBFIREID", "NAT_PARK", "FIRENAME",
    "REP_DATE", "OUT_DATE", "FIRE_TYPE", "RESPONSE",
    "PROTZONE", "MORE_INFO", "MONTH", "DAY"
]

#Using errors="ignore" is intentional because if a columns is already absent
#for example when re-running this cell, we do not want to that it crashs.

fire_df = fire_df.drop(
    columns = COLUMNS_DROP,
    errors="ignore"
).copy()
```

c) Decoding Province Abbrevations

```python
PROVINCE_FULL = {
    "ON": "Ontario",
    "BC": "British Columbia",
    "QC": "Quebec",
    "AB": "Alberta",
    "MB": "Manitoba",
    "SK": "Saskatchewan",
    "NS": "Nova Scotia",
    "NB": "New Brunswick",
    "NL": "Newfoundland and Labrador",
    "PE": "Prince Edward Island",
    "YT": "Yukon",
    "NT": "Northwest Territories",
    "NU": "Nunavut"}

#Ersetzen der Abkürzungen mit dem vollen Namen.
fire_df["province"] = fire_df["province"].map(
    PROVINCE_FULL).fillna(fire_df["province"])
```

d) Decoding Cause Abbrevations

```python
CAUSE_FULL = {
    "H": "Human",
    "H-PB": "Prescribed Burn",
    "U": "Unknown",
    "N": "Natural"}

#Ersetzen der Abkürzungen mit dem vollen Namen.
fire_df["cause"] = fire_df["cause"].map(CAUSE_FULL).fillna(fire_df["cause"])
```

2. Filtering

```python
# Prescribed burns are controlled fire events
# This analysis focuses on uncontrolled fires only.

drop_rows = fire_df[fire_df["cause"]=="Prescribed Burn"].index

fire_df = fire_df.drop(index = drop_rows)
```

```python
#Isolating just the target year (TARGET_YEAR)
fire_2023 = fire_df[fire_df["year"]== TARGET_YEAR]
```

```python
#Remove spatial outliers;
#positive longitudes and negative latitudes are impossible for Canada

fire_2023 = fire_2023[fire_2023["longitude"] <= MAX_VALID_LONGITUDE]
fire_2023 = fire_2023[fire_2023["latitude"] >= MIN_VALID_LATITUDE]

print(f"There were {len(fire_2023)} uncontrolled fires starting in 2023")
```

```
There were 964 uncontrolled fires starting in 2023
```

```python
fire_2023 = fire_2023.sort_values(by="size_ha", ascending=False)
```

3. Classification of fire size

```python
def classify_size(size_ha):
    """
    Classifies a wildfire event by its burned area

    Thresholds follow partly the Canadian wildfire size classification system.
    But since 2023 was a severe year they were a bit adapted so that the count
    of fires in each class is well distributed.

    As Thresholds the constants from Step 0 (SMALL_HA, MEDIUM_HA,
    BIG_HA, and VERY_BIG_HA) will be used

    Parameters
    ----------
    size_ha: Burned area in hectars

    Returns
    -------
    One of: "small", "medium", "big", "very big", "mega fire".
    """
    if size_ha <= SMALL_HA:
        return "small"
    elif 500 <= size_ha < MEDIUM_HA:
        return "medium"
    elif 1000 <= size_ha < BIG_HA:
        return "big"
    elif 10000 <= size_ha < VERY_BIG_HA:
        return "very big"
    else:
        return "mega fire"

# Application of classification
fire_2023["size_class"] = fire_2023["size_ha"].apply(classify_size)
```

```python
#Use an ordered Categorical so for later legends respect the logically set up
#sequence and noth sorting it alphabetically

fire_2023["size_class"] = pd.Categorical(
    fire_2023["size_class"],
    categories=CATEGORY_ORDER,
    ordered=True
)

print(fire_2023["size_class"].value_counts().sort_index())
```

```
size_class
small        203
medium       131
big          370
very big     224
mega fire     36
Name: count, dtype: int64
```

4. Weighting

```python
#Adding predefined class weights of Step 0

fire_2023["class_weight"] = fire_2023["size_class"].map(CLASS_WEIGHTS)
```

5. Export of processed data

```python
fire_2023.to_csv(PROCESSED_PATH, index=False)

print(f"Saved {len(fire_2023):} records to {PROCESSED_PATH}")
```

```
Saved 964 records to ..\data\processed\fire_2023_clean.csv
```

You have now successfully cleaned the data and can proceed with the next notebook about Canadas most populated cities.